

DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

Kishore S (VTU24452)

10211CS224

Full Stack Application Development

Slot : E3-B9



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “Design And Develop a React JS Based Web Application For Ticket Booking Of An Internal Department Event” by Kishore S (VTU24452) has been carried out in the Winter Semester of the Academic Year 2025–2026 (WS2526).

Signature of Faculty

Dr.Hafizur Rahman

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra. M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Kishore S)

Date:06/05/2026

ABSTRACT

Organizing and managing internal department events in academic and corporate institutions often relies on manual ticketing systems, which are prone to errors, double-booking, and inefficient record-keeping. This project presents 'EventMaster,' a dynamic, serverless React JS web application designed specifically for seamless ticket booking and event management. The platform utilizes a modern Glassmorphism User Interface to provide an engaging user experience, while leveraging the HTML5 LocalStorage API as a lightweight, zero-configuration persistence layer. This architecture eliminates the need for complex backend databases while maintaining robust state management across sessions. The system features multi-role authentication, allowing standard users to browse events, book tickets, and view their purchase history, while administrators are equipped with a comprehensive dashboard to launch new events, monitor real-time capacity, and manage a master booking ledger. The resulting application demonstrates a highly portable, efficient, and user-centric approach to full-stack frontend application development.

Keywords: HyperText Markup Language(HTML) , Application Programming Interface (API) , React Js , Glassmorphism , Local Storage , Event Master .

LIST OF FIGURES

1.1	Framework of Ticket Booking Management System	3
3.1	Backend Implementation of Ticket Booking System	9
4.1	Project Architecture of Ticket Booking Management System	11
4.2	Workflow Block Diagram.....	12
4.3	Data Flow Diagram.....	13
5.1	Landing Page Interface	17
5.2	Event Details.....	18
5.3	Ticket Booking Form.....	19
5.4	Booking Confirmation	20
5.5	Admin Dashboard	20

LIST OF TABLES

2.1	Comparative Analysis of Market Alternatives	5
3.1	System Technology Stack	10
7.1	Mapping of Project to CEP	23
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	24
7.3	SDG Mapping	25

LIST OF ACRONYMS AND ABBREVIATIONS

UI	User Interface
UX	User Experience
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
DB	Database
JPA	Java Persistence API
MVC	Model View Controller
CRUD	Create, Read, Update, Delete

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATIONS IN THE MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.1.1 Environment Setup	6
3.1.2 Structure of the Project	7
3.1.3 Execution Procedure	7
3.2 Backend Implementation	8
3.3 Database Implementation	8
3.3.1 Storage Configuration	8
3.3.2 Schema Structure	9
3.3.3 Tables Created	9

3.4	System Technology Stack Overview	10
4	METHODOLOGIES	11
4.1	Project Architecture	11
4.1.1	Workflow	12
4.2	DataFlow Diagram.....	13
4.3	Module 1:.....	14
4.4	Module 2:.....	14
4.5	Module 3:.....	14
5	RESULTS AND DISCUSSIONS	15
5.1	System Modules and Features.....	15
5.2	System Performance and Discussion	16
5.3	System Interface Visualizations	17
5.3.1	Landing Page Interface.....	17
5.3.2	Event Details.....	17
5.3.3	Ticket Booking Form	17
5.3.4	Booking Confirmation.....	18
5.3.5	Admin Dashboard.....	18
6	CONCLUSION AND FUTURE ENHANCEMENTS	21
6.1	Conclusion.....	21
6.2	Future Enhancements.....	21
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	23
7.1	Mapping of the Project to Complex Engineering Problem (CEP).....	23
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	24
7.3	SDG Mapping	25
8	REFRENECES	26

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid digitalization of administrative processes has transformed how organizations handle logistical tasks. However, internal department events—such as hackathons, guest lectures, and paper presentations—frequently still rely on outdated mechanisms like paper tickets or decentralized Google Forms. These methods lack real-time capacity tracking, secure authentication, and unified data management. This project addresses this gap by developing a dedicated web application using React JS. The application focuses on delivering a premium user experience through responsive design and interactive elements, ensuring that the process of browsing and booking event tickets is as seamless as commercial platforms.

1.2 Aim of the Project

The primary aim of this project is to design, develop, and deploy a lightweight, serverless web application for internal event ticket booking. The project aims to provide a secure authentication system, an intuitive user interface for ticket purchasing, and a robust administrative control panel for event lifecycle management, all without the overhead of maintaining a traditional relational database.

1.3 Scope of the Project

The scope of the Ticket Booking Management System includes developing a scalable and user-friendly application that facilitates online ticket booking for internal events. The system covers functionalities such as displaying event details, collecting user information, validating inputs, booking tickets, and updating ticket availability dynamically. It ensures that users cannot book more tickets than available and provides booking confirmation with complete details.

The project also includes backend integration for storing event and booking data, ensuring data consistency and efficient processing. It is designed to be adaptable for future enhancements such as user authentication, online payment integration, email notifications, and admin modules for event creation and management, making it suitable for real-world applications.

1.4 Framework

The application is built entirely on the React JS framework. It utilizes React Hooks for complex state management and side effects. The styling is accomplished using pure CSS3, specifically implementing a 'Glassmorphism' design system characterized by frosted glass aesthetics, CSS variables for theming, and keyframe animations for dynamic user feedback.

Figure 1.1 shows The framework of a Ticket Booking Management System shows how users, administrators, and the database interact to manage ticket reservations.

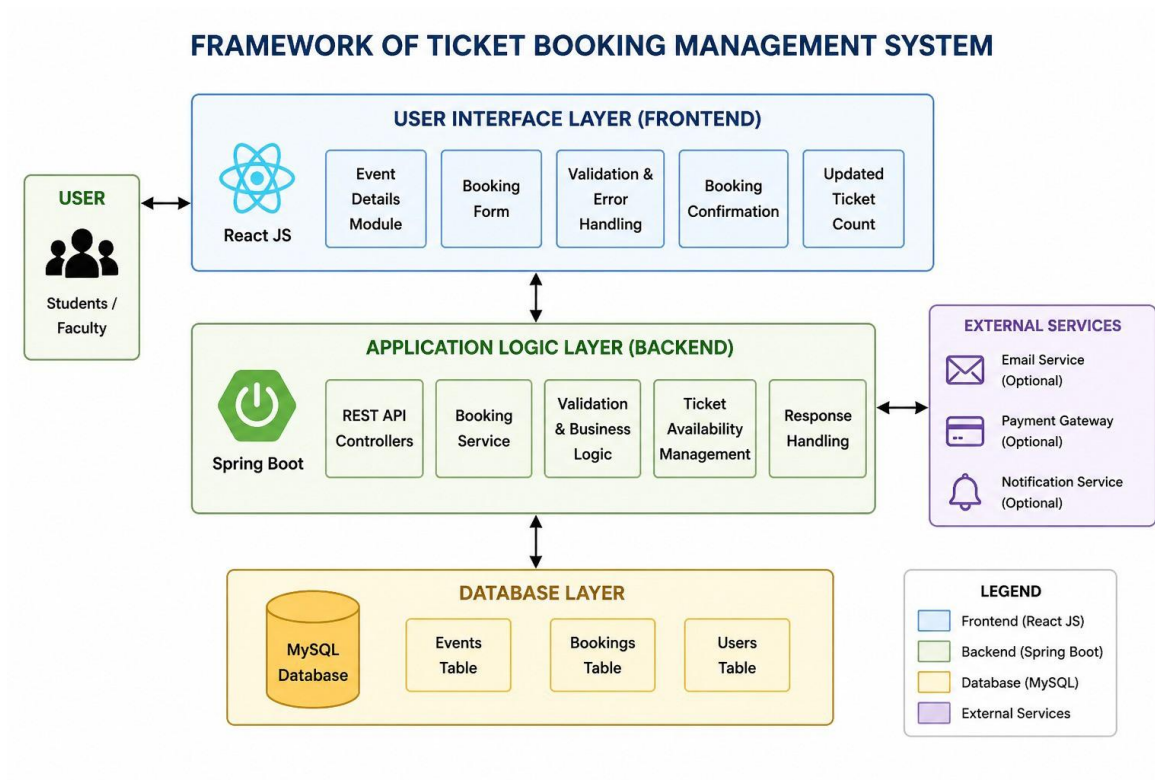


Figure 1.1: Framework of Ticket Booking Management System

Figure 1.1 shows The framework of a Ticket Booking Management System shows how users, administrators, and the database interact to manage ticket reservations.

Chapter 2

SURVEY OF SIMILAR APPLICATIONS IN THE MARKET

In the current digital era, several online ticket booking platforms are widely used to manage and book tickets for events, making the process faster and more convenient. Popular platforms such as BookMyShow, Eventbrite, and Townscript dominate the event management space by providing users with access to a wide range of events including movies, concerts, seminars, and workshops. These platforms allow users to view event details, select tickets, and complete bookings through an easy-to-use interface. They also offer additional features such as seat selection, online payments, e-tickets, and event reminders, enhancing the overall user experience.

Among these platforms, BookMyShow is one of the most widely used applications in India, primarily focused on movie and entertainment ticket booking with features like seat selection and instant confirmation. Eventbrite is known for its global reach and is widely used for managing professional events, conferences, and workshops with tools for event creation and promotion. Townscript is popular for handling technical events, marathons, and college fests, providing simple event registration and ticketing solutions. Each of these platforms focuses on large-scale event management and caters to diverse audiences.

In addition to these major platforms, there are several other event management systems that provide specialized services such as ticketing for niche events, community programs, or corporate gatherings. Many modern systems also integrate advanced features such as analytics, promotional tools, and automated notifications to

improve event management efficiency and user engagement.

Despite their advantages, existing ticket booking systems have certain limitations such as lack of focus on internal department events, limited customization for small-scale academic use, and dependency on online payment systems. These platforms are often designed for large public events and may not efficiently support small or internal institutional requirements.

The proposed Ticket Booking Management System aims to address these limitations by providing a simple and efficient platform specifically designed for internal department events. It focuses on controlled ticket booking, real-time availability updates, validation-based booking, and direct interaction between users and administrators. This makes the system more suitable for academic environments and small-scale event management.

Table 2.1: Comparative Analysis of Market Alternatives

Feature	BookMyShow	Eventbrite	Townscript	Your Project
Event Creation	Yes	Yes	Yes	Yes
Ticket Booking	Yes	Yes	Yes	Yes
Internal Department Events	No	No	No	Yes
User Registration/Login	Yes	Yes	Yes	Optional
Online Payment Integration	Yes	Yes	Yes	No / Planned
E-Ticket	Yes	Yes	Yes	Yes
Seat Selection	Yes	Limited	Limited	Yes
Event Analytics	Yes	Yes	Yes	Basic
Direct Communication (Admin-User)	No	Limited	No	Yes

Table 2.1: This table presents a comparison of different existing ticket booking platforms based on key features such as usability, cost, performance, and functionality

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Ticket Booking Management System is developed using modern web technologies to create a fast, responsive, and user-friendly interface. The application is built using React.js, a component-based JavaScript library that enables dynamic UI rendering. The project is scaffolded and managed using Vite, which provides a fast development server and optimized build process.

Tailwind CSS (v3) is used for designing the user interface with a utility-first approach, allowing quick styling, responsive layouts, and modern UI elements. React Router DOM is used for client-side routing, enabling seamless navigation between pages such as dashboard, event details, booking form, admin panel, login, and confirmation pages without reloading the browser.

Axios is used for making HTTP requests to communicate with the backend APIs. Lucide React is used to integrate modern vector icons into the application, improving visual appearance. React Hot Toast is used for displaying real-time notifications such as booking success and error messages. LocalStorage is used to store authentication tokens and user preferences, ensuring persistence across sessions.

3.1.2 Structure of the Project

The frontend project follows a modular structure for better scalability and maintainability:

- **/src:** Main source folder containing all React components and logic.
- **/components:** Reusable UI components such as Navbar, Event Card, Booking Form, and Buttons.
- **/App.js:** Contains main pages like Dashboard, Event Details, Admin Panel, Login, and Booking Success page.
- **/App.css:** Defines the glassmorphism design system using CSS variables for dark/light mode support.
- **/assets:** Stores images, icons, and static files.
- **/styles:** Contains Tailwind CSS configurations and custom styles.
- **/routes:** Defines application routing using React Router.
- **main.jsx:** Entry point of the React application.
- **package.json:** Manages dependencies and project scripts.

3.1.3 Execution Procedure

Once the server is running, the application can be accessed through a web browser. Users can view events, book tickets, and receive confirmation messages, while administrators can manage events and bookings.

3.2 Backend Implementation

In this project, a traditional backend server is not implemented. Instead, the Ticket Booking Management System operates using client-side storage (Local Storage) provided by the browser.

All application logic, including event management, user handling, and ticket booking, is implemented within the frontend using React.js.

Local Storage is used to store and retrieve data such as:

User details (name, email, password) Event information (event name, date, venue, tickets) Booking records (selected event, number of tickets, total amount)

The application simulates backend functionality by performing operations like:

Adding and retrieving data from Local Storage Validating user inputs Managing booking processes dynamically

Since there is no server-side processing:

No REST APIs are used No database connections are required All data is stored and accessed within the user's browser

This approach simplifies development and is suitable for small-scale or prototype applications, but it has limitations in terms of scalability, security, and multi-user support. APIs are tested using Postman to ensure correctness and reliability.

3.3 Database Implementation

3.3.1 Storage Configuration

Instead of using a traditional database like MySQL, this system uses browser Local Storage to manage data.

Local Storage stores data in key-value pairs, allowing persistent storage even after the browser is refreshed.

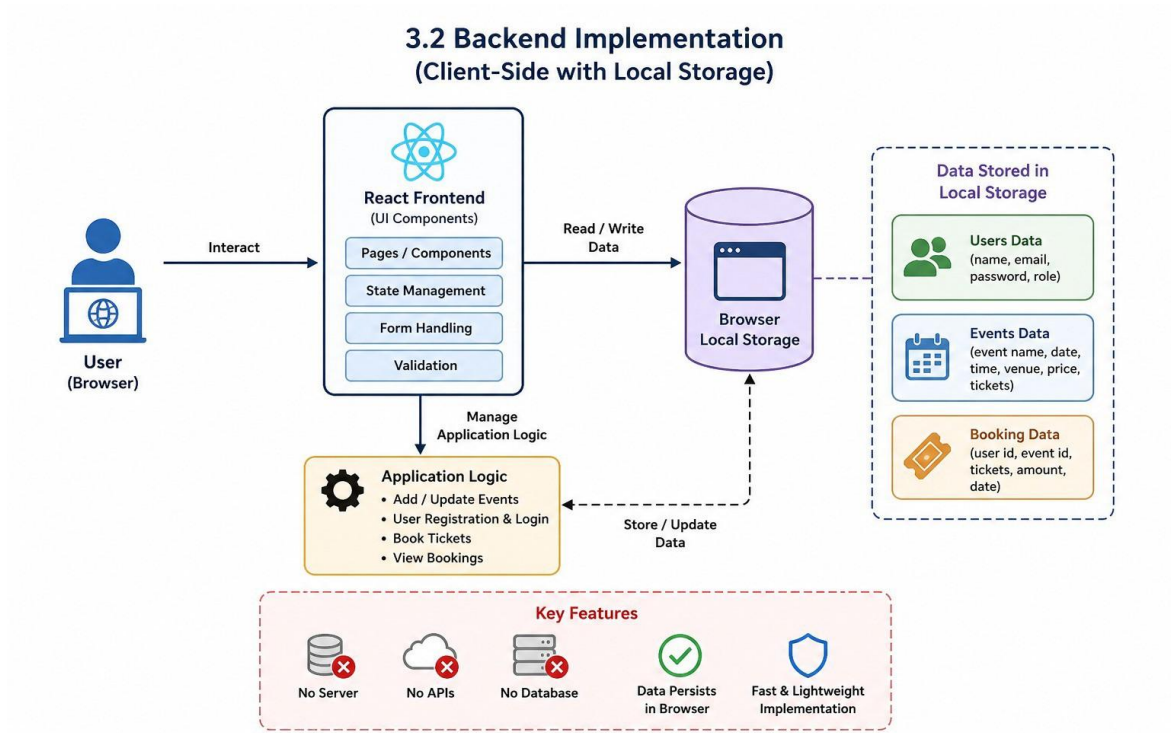


Figure 3.1: Backend Implementation of Ticket Booking System using Local Storage

Figure 3.1: Backend Implementation of Ticket Booking System

Figure 3.1: This figure illustrates the backend architecture of the ticket booking system, showing how server-side components handle user requests, business logic, and database interactions

3.3.2 Schema Structure

The database schema is designed using structured tables with relationships such as One-to-Many (Event to Bookings) and Many-to-One (Bookings to User). This ensures efficient data storage and retrieval.

3.3.3 Tables Created

- **Users:** Stores user details such as name, email, encrypted password, and role (Admin/User).
- **Events:** Contains event details including event name, department, date, time, venue, ticket price, and available tickets.
- **Bookings:** Stores booking details such as booking ID, user ID, event ID, num-

ber of tickets, total amount, and booking date.

3.4 System Technology Stack Overview

Table 3.1: System Technology Stack

Component	Technology
Frontend	React.js, Vite, Tailwind CSS, React Router DOM
Backend	Spring Boot, Spring Web, Spring Data JPA, Hibernate
Database	MySQL
API Communication	REST APIs, Axios
Security	SHA-256, Spring Interceptors
Tools	VS Code, Postman, Git, npm

Table 3.1: This table lists the technologies used to develop the system, including programming languages, frameworks, databases, and tools.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a Single Page Application (SPA) architecture. The central state is maintained in App.js, acting as the single source of truth. State changes trigger React's reconciliation process, updating the DOM dynamically. Side effects are utilized to synchronize the React state with the browser's LocalStorage, ensuring data persists across page reloads.

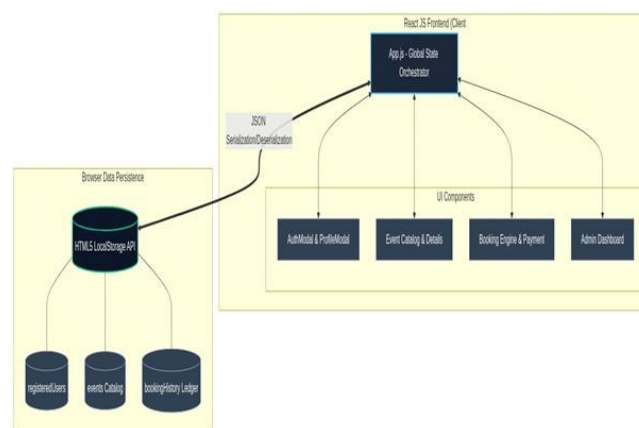


Figure 4.1: Project Architecture of Ticket Booking Management System

Figure 4.1 : This figure illustrates the overall architecture of the ticket booking management system, showing how the frontend, backend, and database components are interconnected.

4.1.1 Workflow

The workflow of the Ticket Booking Management System begins when users access the application through the web interface. Users can view event details and proceed to book tickets by entering required information such as name, email ID, department, and number of tickets.

The request is sent to the backend, where Spring Boot controllers process the input, apply validation rules, and execute booking logic through the service layer. The system checks ticket availability and updates the database accordingly.

Once the booking is successful, a confirmation message along with booking details is generated and sent back to the frontend. The updated ticket count is displayed in real-time, ensuring accurate and smooth interaction between the user and the system.

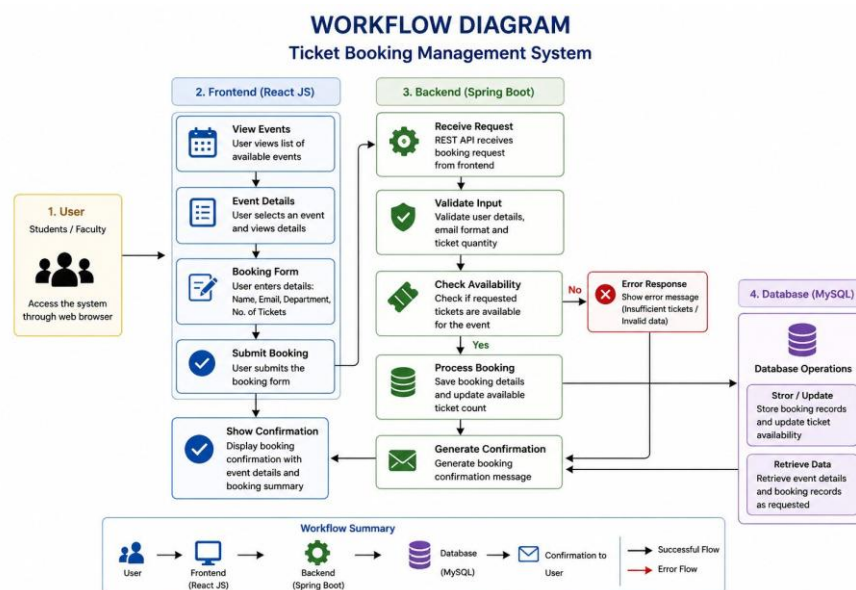


Figure 4.2: Workflow Block Diagram

Figure 4.2 : This figure represents the step-by-step workflow of the system, illustrating how data flows from user input through processing to final output.

4.2 DataFlow Diagram

The sequence of operations moves linearly from User Input React State Orchestration LocalStorage persistence. For example, when an Admin deletes an event, the system instantly filters the master booking ledger, scrubs all associated tickets, saves the cleaned data back to LocalStorage, and re-renders the DOM to show the updated catalog.

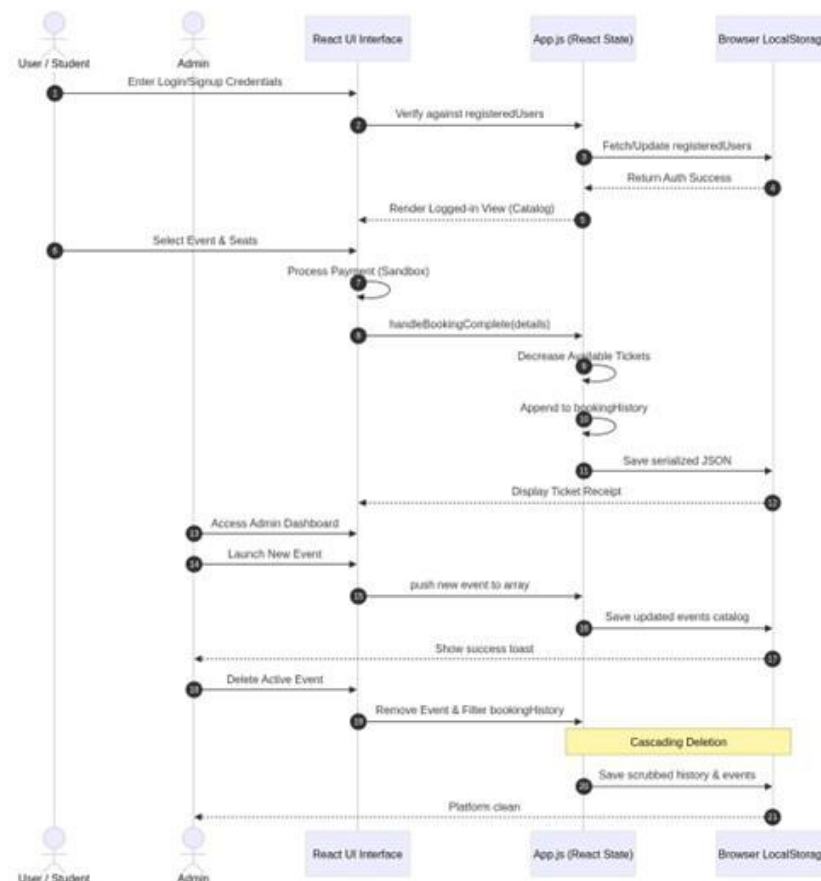


Figure 4.3: Data Flow Diagram

Figure 4.3 :This figure shows how data moves through the system, illustrating the flow of information between users, processes, and the database.

4.3 Module 1:

Authentication User Profiles Handles user registration, login, and session management. It includes privilege escalation, identifying the ‘admin@eventmaster.com’ account and granting it special UI access. The module also allows users to update their credentials or permanently delete their accounts, which triggers a cascading deletion algorithm to wipe their tickets and restore event capacities.

4.4 Module 2:

Event Catalog Booking Engine: Displays available events with real-time capacity updates. When a user initiates a booking, the engine dynamically checks seat availability, enforces authentication guards, and processes a simulated secure payment via the ‘PaymentModal’. Upon success, it generates a randomized Ticket ID and appends the transaction to the global ledger.

4.5 Module 3:

Administrative Dashboard : A protected route accessible only by Master Admins. It features a three-tab interface:

- **Manage Events:** Allows full CRUD (Create, Read, Update, Delete) operations on the event catalog. Features safe-delete warnings if an event currently has active bookings.
- **Master Booking Log:** A global ledger viewing all tickets purchased by all users across the platform.
- **Launch Event:** A form to instantly deploy new events into the public catalog.

Chapter 5

RESULTS AND DISCUSSIONS

The Ticket Booking Management System is developed as a full-stack web application that simplifies the process of booking tickets for internal department events. The system integrates multiple functional modules to provide an efficient and user-friendly experience for students, faculty, and administrators.

5.1 System Modules and Features

The implemented system consists of the following key modules:

- **Landing Page Interface:** Provides an overview of available events and system features with easy navigation.
- **Event Details Module:** Displays event information such as event name, department, date, time, venue, ticket price, and available tickets.
- **Ticket Booking Module:** Allows users to book tickets by entering details like name, email, department, and number of tickets.
- **Validation Module:** Ensures all inputs are valid, including mandatory fields, correct email format, and positive ticket quantity.
- **Booking Management Module:** Handles booking records, updates ticket availability, and stores booking details.
- **User Interface Module:** Provides interactive UI components using React JS for smooth user experience.

- **Admin Management Module:** Allows administrators to manage events and bookings efficiently.
- **Notification System:** Displays booking confirmation messages and optionally sends email notifications.
- **Database Layer:** Stores event details, user information, and booking records using a relational database.

5.2 System Performance and Discussion

The system follows a modular and layered architecture that ensures scalability and efficient handling of booking operations. It enables smooth interaction between users and the system while maintaining data consistency and real-time updates.

The performance of the system is evaluated based on responsiveness, data handling, and user interaction. The application provides quick response times during event viewing, ticket booking, and data retrieval.

The system prevents overbooking by validating ticket availability and ensures accurate data handling through proper validation mechanisms. The integration between React JS frontend and Spring Boot backend ensures seamless communication.

The user interface is simple and intuitive, allowing users to easily navigate through event details and booking processes. The admin module provides efficient control for managing events and bookings.

Overall, the system reduces manual effort, minimizes errors, and improves efficiency in event management.

5.3 System Interface Visualizations

5.3.1 Landing Page Interface

The landing page Fig. 5.1 provides an overview of available events and system features.

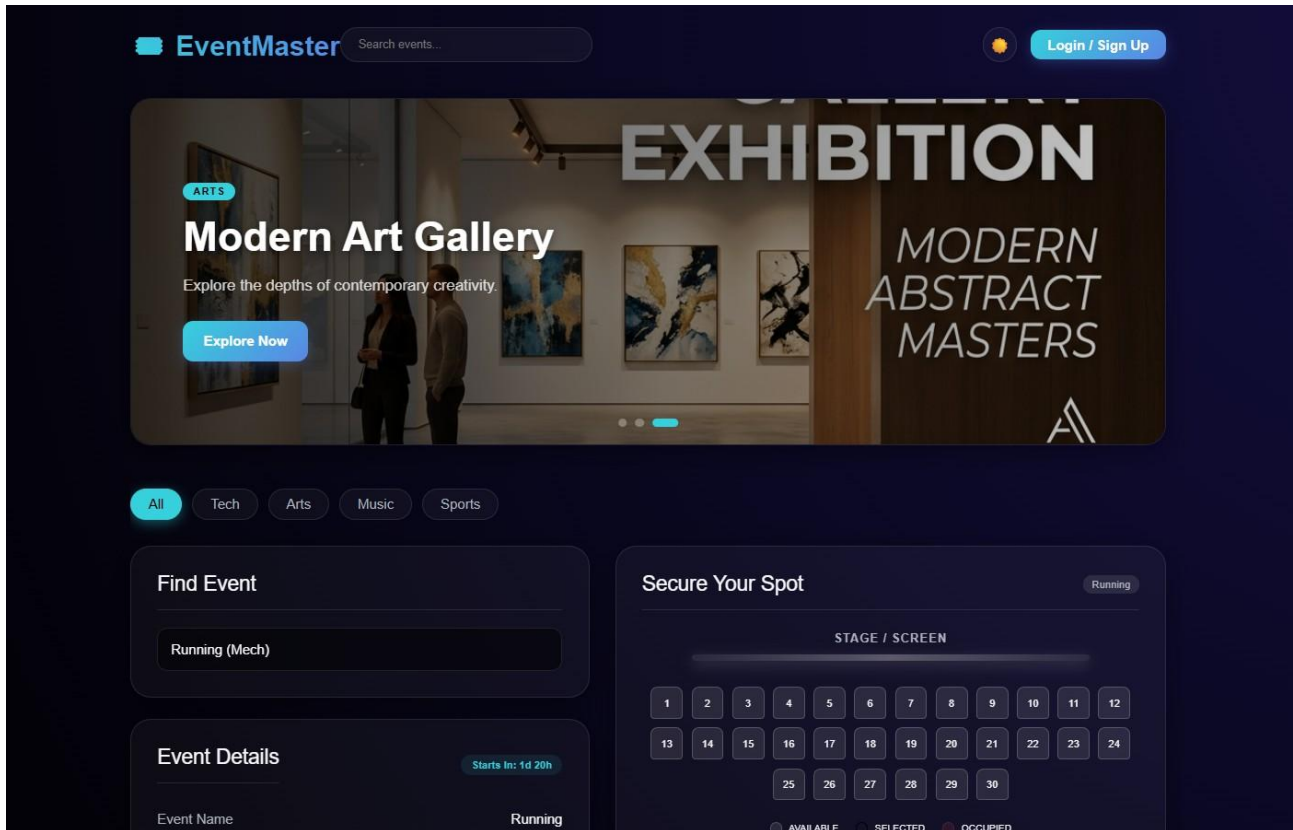


Figure 5.1: Landing Page Interface

5.3.2 Event Details

The event details module Fig. 5.2 displays complete information about events.

5.3.3 Ticket Booking Form

The booking form Fig. 5.3 allows users to enter booking details.

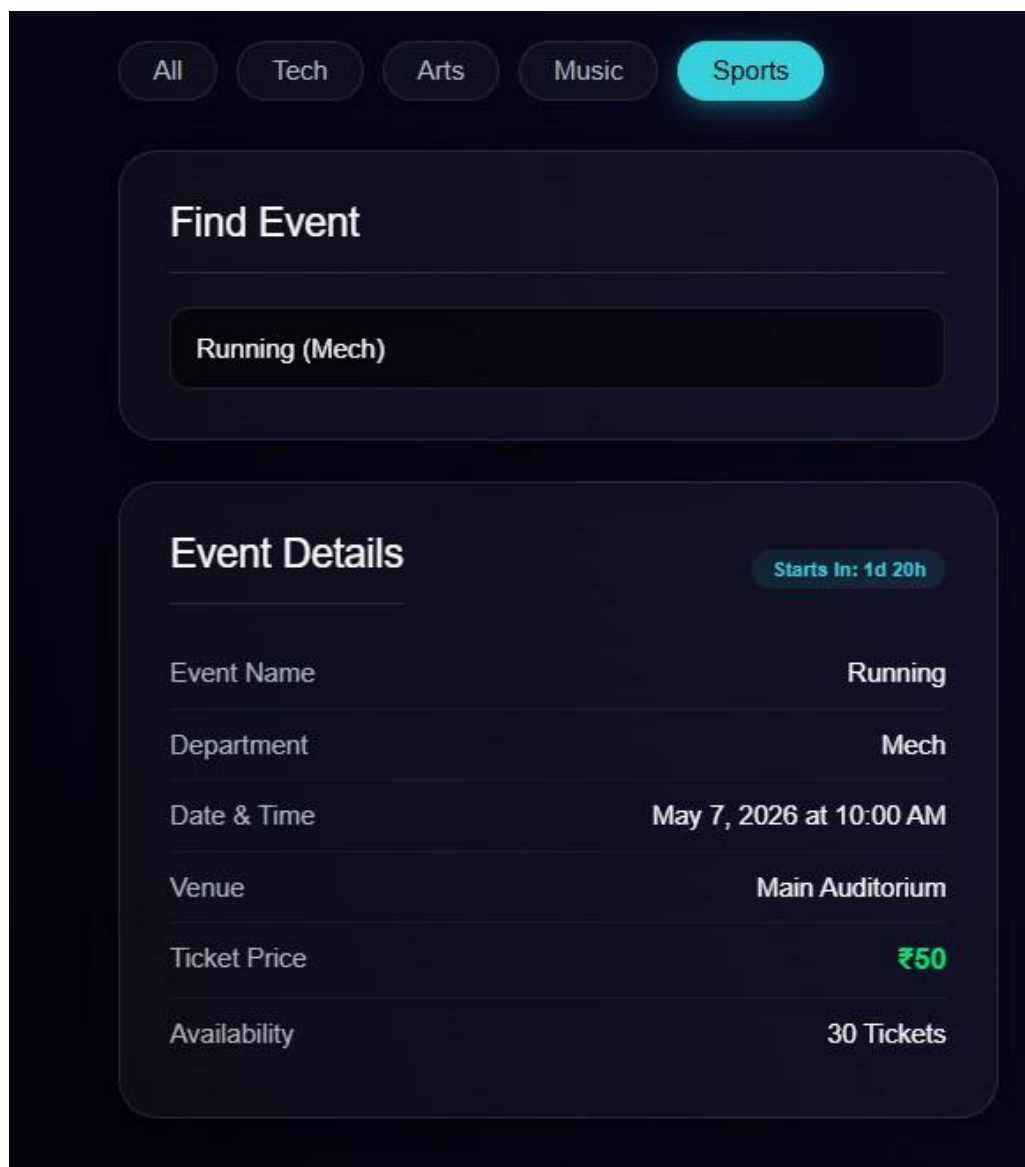


Figure 5.2: Event Details

5.3.4 Booking Confirmation

The confirmation module Fig. 5.4 displays booking summary after successful booking.

5.3.5 Admin Dashboard

The admin dashboard Fig. 5.5 shows event and booking statistics.

Secure Your Spot

Running

STAGE / SCREEN

123456789101112

131415161718192021222324

252627282930

AVAILABLE

SELECTED

OCCUPIED

Passenger Details

Name for Seat 6

Name for Seat 1

Contact Email Address

Department / Organization

Selected: 2 ticket(s)

Total: ₹100

Reset

Confirm Booking

Figure 5.3: Ticket Booking Form

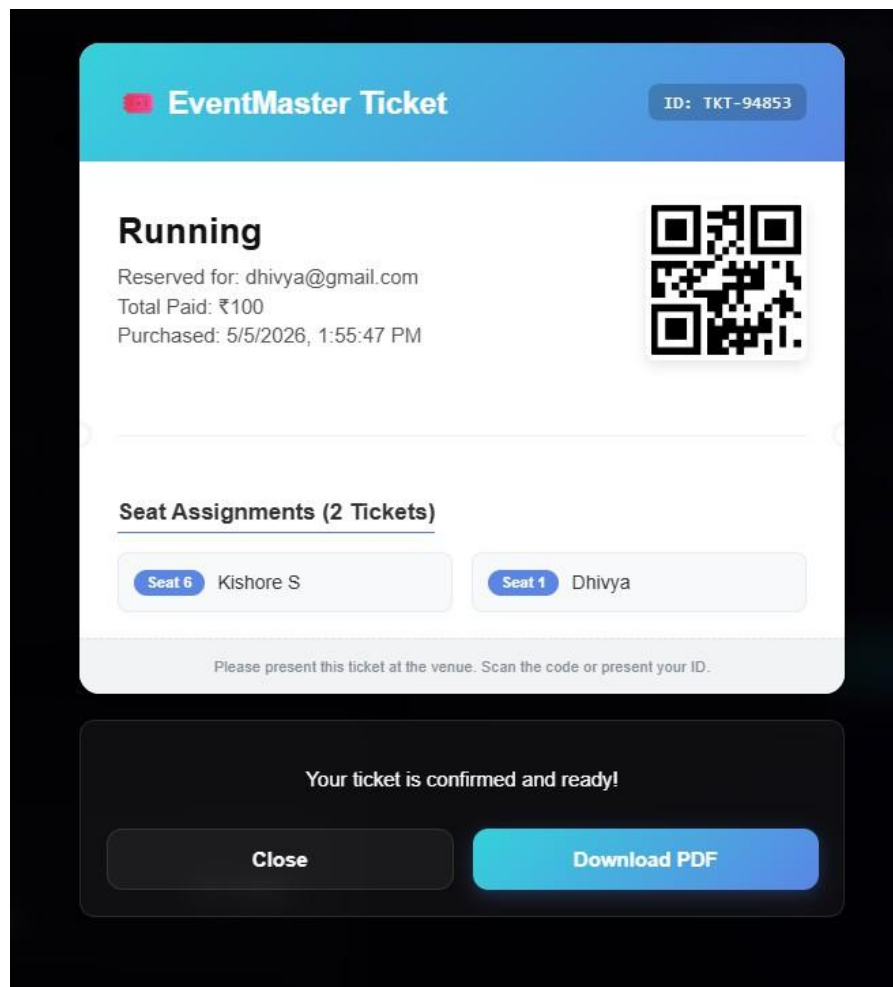


Figure 5.4: Booking Confirmation

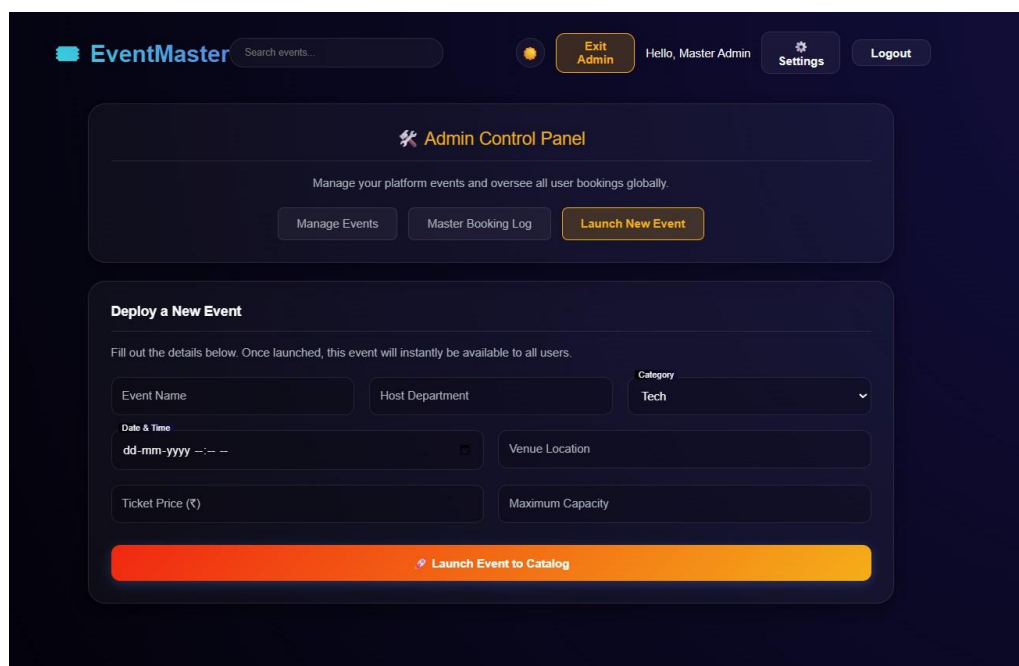


Figure 5.5: Admin Dashboard

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The EventMaster project demonstrates the sheer power and capability of modern React JS frontend development. By creatively utilizing client-side storage mechanisms, the project achieved a fully functional, state-driven application capable of handling complex relational data logic (such as cascading deletions and capacity tracking) entirely within the browser. The intuitive user interface and robust administrative controls provide a complete, end-to-end solution for managing internal department events effectively.

6.2 Future Enhancements

While the current system provides a strong and functional foundation, several enhancements can be implemented to further improve its capabilities and usability:

- **Online Payment Integration:** Integrate secure payment gateways such as UPI, credit/debit cards, or net banking to enable users to complete ticket booking transactions online in a safe and convenient manner.
- **Email Notifications:** Implement automated email notifications to send booking confirmations, event reminders, and important updates to users, improving communication and user engagement.

- **Mobile Application:** Develop a mobile application version of the system to provide better accessibility and allow users to book tickets and receive updates on-the-go.
- **QR Code E-Tickets:** Generate QR code-based electronic tickets for each booking, which can be scanned at the event venue for quick and secure verification.
- **Advanced Analytics Dashboard:** Provide detailed analytics and reports for administrators, including insights on ticket sales, event popularity, and user participation trends.
- **User Authentication System:** Implement a secure login and registration system with role-based access control to enhance data security and provide personalized user experiences.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to CEP

Level	Attribute	Characteristics	Justification
WP1	Depth of Knowledge	Engineering knowledge	Uses React, Spring Boot, APIs, DB design
WP2	Conflicting Requirements	Tech vs usability	Balances usability and validation
WP3	Depth of Analysis	Design thinking	Includes booking logic
WP4	Familiarity	Partially new problems	Handles real-time updates
WP5	Applicable Codes	Limited standards	Custom system design
WP6	Stakeholder Needs	Multiple users	Supports users and admins
WP7	Interdependence	System integration	Integrates frontend, backend, DB

Table 7.1 : This table shows how different components of the project align with the Course Educational Program (CEP) outcomes.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

Table 7.2 :illustrates how the project satisfies various Complex Engineering Activity (CEA)

7.3 SDG Mapping

Table 7.3: SDG Mapping

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Supports academic events, seminars, and technical fests, helping students gain knowledge and practical exposure.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation by implementing a modern web-based ticket booking system.
SDG 11	Sustainable Cities and Communities	Encourages efficient digital event management and reduces dependency on manual systems.
SDG 12	Responsible Consumption and Production	Reduces paper usage by replacing physical tickets with digital booking and e-tickets.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure access, proper validation, and reliable system operations.

Table 7.3 : This table illustrates how the project aligns with relevant United Nations Sustainable Development Goals (SDGs)

REFERENCES

- [1] A. Hemrajani, R. Balpande, and P. Dar, “BookMyShow: Online Ticket Booking Platform,” Available: <https://in.bookmyshow.com>

- [2] J. Hartz, K. Hartz, and R. Visage, “Eventbrite: Event Management and Ticketing Platform,” Available: <https://www.eventbrite.com>

- [3] S. Sharma and S. Malik, “Townscript: Event Registration Platform,” Available: <https://www.townscript.com>

- [4] J. Walke, “React JS Documentation,” Available: <https://reactjs.org>

- [5] R. Johnson and Pivotal Software Team, “Spring Boot Documentation,”